Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Пермский институт (филиал) РЭУ им. Г.В. Плеханова

Практическая работа

МДК _____ / ОП ____
«Технология разработки программного обеспечения»

Специальность **09.02.07**
«Информационные системы и программирование»

Листов: 25

Руководитель
Дмитрий Борисович Берестов
«_____» _____ 2026 г.

Исполнитель
Лука Евгеньевич Катаев
«_____» _____ 2026г.

2026

Оглавление

Создание роли, базы и настройка прав доступа 3
Создание таблиц 5
Схема базы данных 7
Создание библиотеки классов 8
Диаграмма классов 17
Создание WPF приложения 18
Создание модульных тестов 23
Вывод 25

Создание роли, базы и настройка прав доступа

Для начала нам необходимо установить PostgreSQL, конечно, если он не был установлен, также его можно установить через официальный сайт.

Теперь переходим к созданию роли, для этого открываем psql shell и пишем команду об создании новой роли.

Перед созданием базы данных нам необходимо создать роль, с помощью которой мы зайдем в саму базу и будем ее редактировать. Для это переходим в SQL Shel и создаем новую роль с помощью команды CREATE ROLE с параметром login. Отлично, мы создали роль, присвоим роли пароль, для этого прописываем \password вводим и подтверждаем пароль.

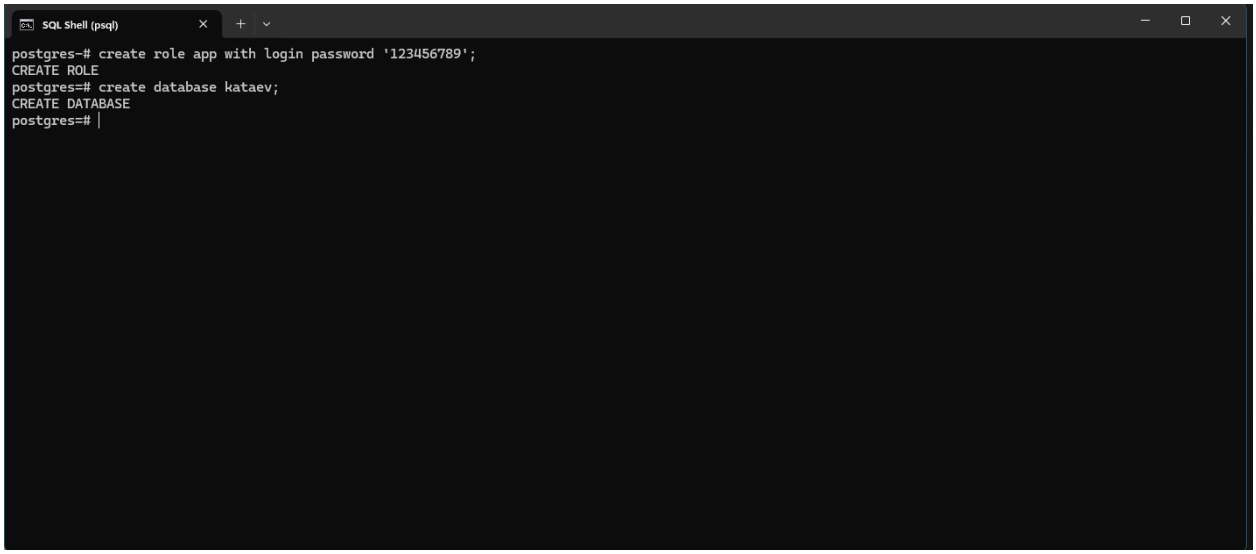

Рисунок 1 создание роли

После чего создаем базу данных используя, команду CREATE DATABASE, отлично, мы создали базу данных, теперь можно подключиться, используем для этого пользователя postgres. Как только мы зашли, нам необходимо предоставить все права нашей ранее созданной роли. Для этого используем команду ALTER DEFAULT PRIVILEGES IN SCHEMA app, при этом используем необходимые нам параметры, которые в дальнейшем позволят нам создавать, редактировать, удалять таблицы, работать со схемами, а также создавать и использовать процедуры и функции.

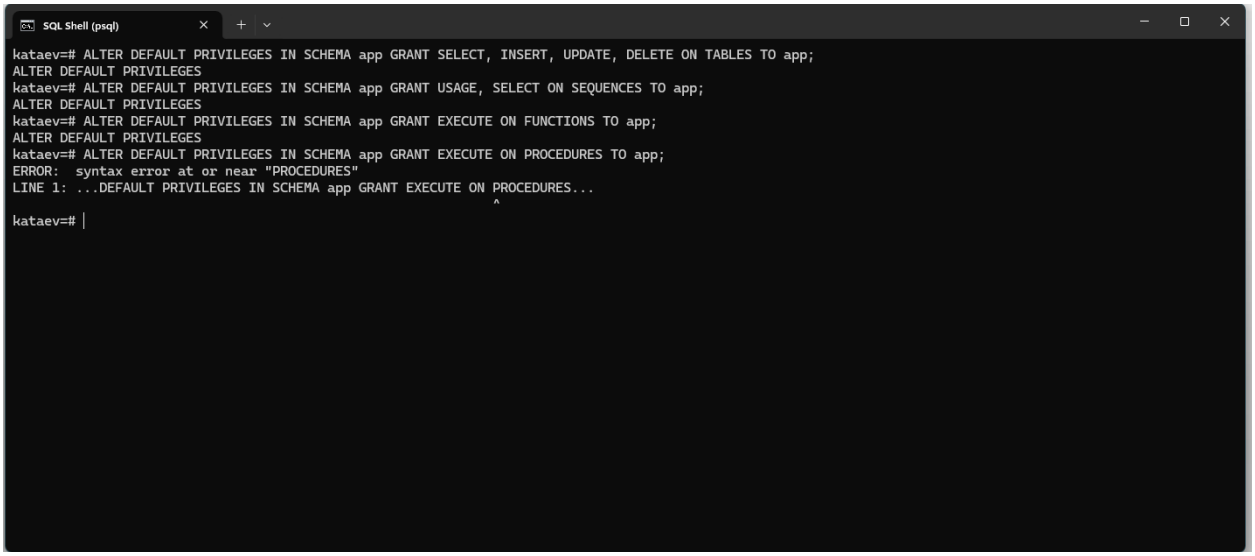

Рисунок 2 выдача прав

Как только мы выдали все необходимые права, мы можем подключиться к нашей базе данных от имени ранее созданной роли.

Создание таблиц

Подключившись от имени нужной нам роли, создаем новую схему app с нашей ролью как параметр для авторизации. Схему мы создали, но нам главное не забыть, что новосозданная схема должна использоваться как основная. И для этого мы устанавливаем в переменную search_path как первое значение – нашу схему app, а как второе значение – автоматически созданная схема public. Данная команда работает следующим образом: она задает порядок схем, то есть схема, которая стоит в начале будет использоваться как основная, но если этой схемы не будет, то в таком случае задается схема public.

Мы завершили подготовительную часть и можем приступить к созданию самих таблиц. Нам необходимо создать следующие таблицы для подсистемы «Учёт партнёров»: таблица KataevPartners (Партнёры), таблица KataevProducts (Продукция) и таблица KataevSalesHistory (История продаж). Рассмотрим каждую таблицу отдельно и отметим уникальные особенности. Начнем с таблицы партнёров. Данная таблица имеет, как и все таблицы, уникальный идентификатор KataevId с типом serial, который авто-инкрементирует каждую новую запись, также мы указываем первичный ключ – Primary Key. У партнёра должно быть наименование, тип организации, рейтинг, адрес, ИНН, ФИО директора (разбито на фамилию, имя и отчество), контактный телефон и электронную почту.

Для текстовых полей используем тип varchar с соответствующей длиной. Таблица с продукцией имеет следующие атрибуты, а именно: уникальный идентификатор KataevId, наименование продукции, её артикул и тип. Первичным ключом также является KataevId. Данная таблица служит справочником для формирования заказов. Таблица истории продаж реализует связь между партнёрами и продукцией. Она имеет свой уникальный идентификатор, а также поля KataevPartnerId и KataevProductId, которые являются внешними ключами (Foreign Key). Они ссылаются на соответствующие таблицы с каскадным удалением (ON DELETE CASCADE), что обеспечивает целостность данных. Также имеется поле количества

KataevQuantity с проверкой (CHECK), что значение больше нуля, и дата продажи KataevSaleDate. Связи между таблицами определены отношением «один-ко-многим» (один партнёр может иметь много записей в истории продаж).

Также не забываем проставить индексы, это нужно для улучшения производительности выборок по внешним ключам и датам. Вот собственно пример создания индекса для таблицы продаж: CREATE INDEX idx_kataev_saleshistory_partnerid ON app."KataevSalesHistory" ("KataevPartnerId"). Аналогичные индексы созданы для поля продукта, даты продажи, а также для ИНН партнёра для ускорения поиска контрагентов.

```
CREATE TABLE app."KataevPartners" (
    "KataevId"                  serial          PRIMARY KEY,
    "KataevName"                varchar(255)    NOT NULL,
    "KataevPartnerType"         varchar(100),
    "KataevRating"              integer         NOT NULL DEFAULT 0,
    "KataevAddress"             varchar(500),
    "KataevINN"                 varchar(12),
    "KataevDirectorLastName"    varchar(100),
    "KataevDirectorFirstName"   varchar(100),
    "KataevDirectorMiddleName"  varchar(100),
    "KataevPhone"               varchar(50),
    "KataevEmail"               varchar(255)
);

CREATE TABLE app."KataevProducts" (
    "KataevId"      serial          PRIMARY KEY,
    "KataevName"    varchar(255)    NOT NULL,
    "KataevArticle" varchar(100),
    "KataevType"    varchar(100)
);

CREATE TABLE app."KataevSalesHistory" (
    "KataevId"          serial  PRIMARY KEY,
    "KataevPartnerId"   integer NOT NULL REFERENCES app."KataevPartners"("KataevId") ON DELETE CASCADE,
    "KataevProductId"   integer NOT NULL REFERENCES app."KataevProducts"("KataevId") ON DELETE CASCADE,
    "KataevQuantity"    integer NOT NULL CHECK ("KataevQuantity" > 0),
    "KataevSaleDate"    date    NOT NULL DEFAULT CURRENT_DATE
);

CREATE INDEX idx_kataev_saleshistory_partnerid  ON app."KataevSalesHistory" ("KataevPartnerId");
CREATE INDEX idx_kataev_saleshistory_productid  ON app."KataevSalesHistory" ("KataevProductId");
CREATE INDEX idx_kataev_saleshistory_saledate   ON app."KataevSalesHistory" ("KataevSaleDate");
CREATE INDEX idx_kataev_partners_inn            ON app."KataevPartners"     ("KataevINN");
```

Рисунок 3 Создание таблиц

Схема базы данных

Для создания ER-схемы базы данных нам требуется зайти в PGAdmin4, в нашей базе выбрать нужную схему, в нашем случае схема app, и нажать на кнопку “ERD for schema”

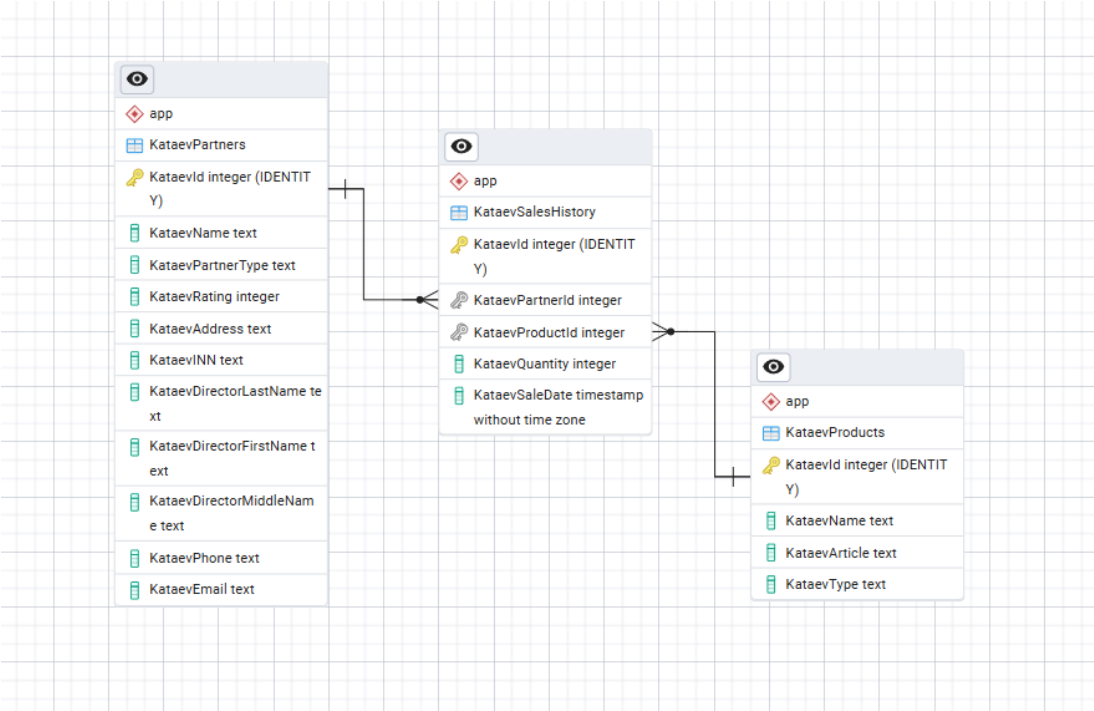

Рисунок 4 схема базы данных

Создание библиотеки классов

Перед началом разработки необходимо выполнить ряд подготовительных шагов, без которых дальнейшая работа с базой данных будет невозможна.

Создание библиотеки классов

Первым шагом создаётся проект библиотеки классов KataevLIB. Именно в ней будет сосредоточена вся бизнес-логика приложения, отделённая от пользовательского интерфейса.

Установка Entity Framework Core

После создания библиотеки необходимо установить пакет Entity Framework Core — объектно-реляционный маппер, обеспечивающий взаимодействие приложения с базой данных без написания SQL-запросов вручную.

Для этого нажимаем правой кнопкой мыши на решение в обозревателе решений и выбираем пункт «Управление пакетами NuGet для решения». В открывшемся окне переходим на вкладку «Обзор» и устанавливаем следующий пакет Npgsql.EntityFrameworkCore.PostgreSQL.

Создание контекста базы данных

Следующим шагом создаётся файл ApplicationContext.cs. Этот класс является центральным элементом работы с базой данных — он наследуется от DbContext и описывает все таблицы, с которыми будет работать приложение. Внутри класса с помощью свойств DbSet<T> объявляются наборы данных, соответствующие таблицам базы данных: KataevPartners для хранения партнёров, KataevProducts для справочника продукции и KataevSalesHistories для истории продаж.

В конструкторе класса вызывается метод Database.EnsureCreated(), который при первом запуске приложения автоматически создаёт все таблицы в базе данных, если они ещё не существуют. Это избавляет от необходимости выполнять миграции вручную.

Метод OnConfiguring отвечает за настройку подключения к базе данных. В нём вызывается optionsBuilder.UseNpgsql — метод, указывающий Entity Framework Core использовать PostgreSQL в качестве СУБД. В строке подключения задаются все необходимые параметры: хост (localhost), порт (5432), имя базы данных (kataev), имя пользователя (app) и пароль.

```
using Microsoft.EntityFrameworkCore;

namespace KataevLIB
{
    Ссылок: 8
    public class ApplicationContext : DbContext
    {
        Ссылок: 6
        public DbSet<KataevPartner> KataevPartners { get; set; }
        Ссылок: 5
        public DbSet<KataevProduct> KataevProducts { get; set; }
        Ссылок: 5
        public DbSet<KataevSalesHistory> KataevSalesHistories { get; set; }

        Ссылок: 1
        public ApplicationContext()
        {
            Database.EnsureCreated();
        }
        Ссылок: 0
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            base.OnConfiguring(optionsBuilder);
            optionsBuilder.UseNpgsql("Host=localhost;Port=5432;Database=kataev;Username=app;Password=123456789;");
        }
        Ссылок: 0
        public ApplicationContext(DbContextOptions<ApplicationContext> options) : base(options) { }
    }
}
```

Рисунок 5 файл ApplicationContext

Создание моделей данных

После настройки контекста базы данных необходимо создать модели — классы, каждый из которых соответствует отдельной таблице в базе данных. В Entity Framework Core такой подход называется Code-First: сначала описываются классы в коде, а фреймворк самостоятельно создаёт по ним таблицы.

Модель KataevPartner

Создаётся новый файл KataevPartner.cs. Класс описывает сущность партнёра компании и отображается на таблицу KataevPartners схемы app — это задаётся атрибутом [Table("KataevPartners", Schema = "app")] над объявлением класса.

Каждое свойство класса соответствует столбцу таблицы. Атрибут [Column] указывает точное имя столбца в базе данных, что необходимо поскольку имена свойств в C# и имена столбцов в БД различаются. Свойство Id помечается атрибутом [Key], обозначающим его как первичный ключ.

Класс содержит одиннадцать полей: идентификатор, наименование организации, тип партнёра, рейтинг, адрес, ИНН, фамилию, имя и отчество директора, телефон и электронную почту.

В конце класса объявляется навигационное свойство KataevSalesHistories типа ICollection<KataevSalesHistory>. Оно помечено ключевым словом virtual, что позволяет Entity Framework Core автоматически подгружать связанные записи истории продаж при обращении к партнёру. Это и есть реализация связи один-ко-многим на уровне кода.

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace KataevLIB
{
    [Table("KataevPartners", Schema = "app")]
    Ссылок: 64
    public class KataevPartner
    {
        [Key]
        [Column("KataevId")]
        Ссылок: 48 | 18/18 passing
        public int Id { get; set; }

        [Column("KataevName")]
        Ссылок: 43 | 17/17 passing
        public string Name { get; set; }

        [Column("KataevPartnerType")]
        Ссылок: 7 | 1/1 passing
        public string PartnerType { get; set; }

        [Column("KataevRating")]
        Ссылок: 9 | 2/2 passing
        public int Rating { get; set; }

        [Column("KataevAddress")]
        Ссылок: 6
        public string Address { get; set; }

        [Column("KataevINN")]
        Ссылок: 7 | 1/1 passing
        public string INN { get; set; }

        [Column("KataevDirectorLastName")]
        Ссылок: 7 | 1/1 passing
        public string DirectorLastName { get; set; }

        [Column("KataevDirectorFirstName")]
        Ссылок: 6 | 1/1 passing
        public string DirectorFirstName { get; set; }

        [Column("KataevDirectorMiddleName")]
        Ссылок: 5
        public string DirectorMiddleName { get; set; }

        [Column("KataevPhone")]
        Ссылок: 6
        public string Phone { get; set; }

        [Column("KataevEmail")]
        Ссылок: 6
        public string Email { get; set; }

        Ссылок: 2
        public virtual ICollection<KataevSalesHistory> KataevSalesHistories { get; set; }
    }
}
```

Рисунок 6 Модель KataevPartner

Модель KataevProduct

По аналогии создаётся файл KataevProduct.cs. Класс отображается на таблицу KataevProducts и содержит четыре поля: идентификатор, наименование продукции, артикул и тип. Также содержит навигационное свойство KataevSalesHistories для связи с историей продаж.

```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace KataevLIB
{
    [Table("KataevProducts", Schema = "app")]
    Ссылок: 30
    public class KataevProduct
    {
        [Key]
        [Column("KataevId")]
        Ссылок: 9 | 2/2 passing
        public int Id { get; set; }

        [Column("KataevName")]
        Ссылок: 14 | 5/5 passing
        public string Name { get; set; }

        [Column("KataevArticle")]
        Ссылок: 11 | 2/2 passing
        public string Article { get; set; }

        [Column("KataevType")]
        Ссылок: 11 | 3/3 passing
        public string Type { get; set; }

        Ссылок: 0
        public virtual ICollection<KataevSalesHistory> KataevSalesHistories { get; set; }
    }
}
```

Рисунок 7 Модель KataevProduct

Модель KataevSalesHistory

Создаётся файл KataevSalesHistory.cs. Этот класс является связующим между партнёром и продукцией — он отображается на таблицу KataevSalesHistory и реализует отношение многие-ко-многим через две отдельные связи один-ко-многим.

Класс содержит пять полей: идентификатор записи, идентификатор партнёра (PartnerId), идентификатор продукции (ProductId), количество реализованных единиц и дату продажи.

Поля PartnerId и ProductId являются внешними ключами — это указывается атрибутом [ForeignKey]. Помимо самих ключей, класс содержит навигационные свойства KataevPartner и KataevProduct, через которые Entity Framework Core позволяет напрямую обращаться к связанным объектам без написания JOIN-запросов вручную.

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace KataevLIB
{
    [Table("KataevSalesHistory", Schema = "app")]
    Ссылок: 38
    public class KataevSalesHistory
    {
        [Key]
        [Column("KataevId")]
        Ссылок: 5
        public int Id { get; set; }

        [Column("KataevPartnerId")]
        Ссылок: 20 | 7/7 passing
        public int PartnerId { get; set; }

        [Column("KataevProductId")]
        Ссылок: 5
        public int ProductId { get; set; }

        [Column("KataevQuantity")]
        Ссылок: 22 | 7/7 passing
        public int Quantity { get; set; }

        [Column("KataevSaleDate")]
        Ссылок: 6 | 1/1 passing
        public DateTime SaleDate { get; set; }

        [ForeignKey("PartnerId")]
        Ссылок: 0
        public virtual KataevPartner KataevPartner { get; set; }

        [ForeignKey("ProductId")]
        Ссылок: 1
        public virtual KataevProduct KataevProduct { get; set; }
    }
}
```

Рисунок 8 Модель KataevSalesHistory

Создание интерфейса IMainApp и класса MainApp

Интерфейс IMainApp

Создаётся файл IMainApp.cs. Интерфейс объявляет контракт всех операций, которые должна поддерживать бизнес-логика приложения. Его использование позволяет отделить описание операций от их конкретной реализации — WPF-приложение работает именно с интерфейсом, не завися от деталей реализации. Это также упростило написание модульных тестов, где реальная реализация была подменена тестовой.

Интерфейс содержит три группы методов: управление контекстом (GetContext, SaveChanges, SetupAppSchema), CRUD-операции для каждой из трёх сущностей и метод бизнес-логики CalculatePartnerDiscount.

```
public interface IMainApp
{
    Ссылок: 2
    ApplicationContext GetContext();
    Ссылок: 12
    void SaveChanges();
    Ссылок: 3
    void SetupAppSchema();

    Ссылок: 27 | 17/17 passing
    void AddPartner(KataevPartner partner);
    Ссылок: 4 | 1/1 passing
    void UpdatePartner(KataevPartner partner);
    Ссылок: 5 | 2/2 passing
    void RemovePartner(KataevPartner partner);
    Ссылок: 10 | 5/5 passing
    IEnumerable<KataevPartner> GetPartners();
    Ссылок: 5 | 3/3 passing
    KataevPartner GetPartner(int partnerId);
    Ссылок: 4 | 2/2 passing
    IEnumerable<KataevPartner> GetPartners(string searchTerm);

    Ссылок: 9 | 5/5 passing
    void AddProduct(KataevProduct product);
    Ссылок: 4 | 1/1 passing
    void UpdateProduct(KataevProduct product);
    Ссылок: 4 | 1/1 passing
    void RemoveProduct(KataevProduct product);
    Ссылок: 8 | 4/4 passing
    IEnumerable<KataevProduct> GetProducts();
    Ссылок: 5 | 3/3 passing
    KataevProduct GetProduct(int productId);

    Ссылок: 15 | 7/7 passing
    void AddSalesHistory(KataevSalesHistory salesHistory);
    Ссылок: 4 | 1/1 passing
    void UpdateSalesHistory(KataevSalesHistory salesHistory);
    Ссылок: 4 | 1/1 passing
    void RemoveSalesHistory(KataevSalesHistory salesHistory);
    Ссылок: 3 | 1/1 passing
    IEnumerable<KataevSalesHistory> GetSalesHistory(KataevPartner partner);
    Ссылок: 12 | 6/6 passing
    IEnumerable<KataevSalesHistory> GetSalesHistory(int partnerId);

    Ссылок: 12 | 9/9 passing
    int CalculatePartnerDiscount(int partnerId);
}
```

Рисунок 9 Интерфейс IMainApp

Класс MainApp

Создаётся файл MainApp.cs. Класс реализует интерфейс IMainApp, а также IDisposable для корректного освобождения контекста базы данных по завершении работы.

В конструкторе вызывается метод SetupAppSchema, который при каждом запуске убеждается в существовании схемы app и выдаёт роли app необходимые права доступа через прямое выполнение SQL-команд.
CRUD-методы для партнёров, продукции и истории продаж делегируют операции соответствующим DbSet контекста. Метод AddSalesHistory дополнительно проверяет существование партнёра и выбрасывает исключение если он не найден. Методы получения данных используют Include для загрузки связанных сущностей.

Ключевым методом является CalculatePartnerDiscount — он суммирует количество всей реализованной продукции партнёра и возвращает процент скидки согласно пороговым значениям: до 10 000 — 0%, до 50 000 — 5%, до 300 000 — 10%, свыше — 15%.

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;

namespace KataevLIB
{

    public class MainApp : IDisposable, IMainApp
    {

        public ApplicationContext Context { get; private set; } = new ApplicationContext();
        private bool disposedValue;

        public void Dispose()
        {
            Dispose(true);
            GC.SuppressFinalize(this);
        }

        protected virtual void Dispose(bool disposing)
        {
            if (!disposedValue)
            {
                if (disposing)
                {
                    Context.Dispose();
                }

                disposedValue = true;
            }
        }


        public MainApp()
        {
            SetupAppSchema();
        }


        public ApplicationContext GetContext()
        {
            return this.Context;
        }
```

Рисунок 10 Класс MainApp

Диаграмма классов

Для создания диаграммы классов переходим во вкладку проект, выбираем добавить новый элемент и ищем диаграмму классом. После ее создания, перетаскиваем наши файлы.

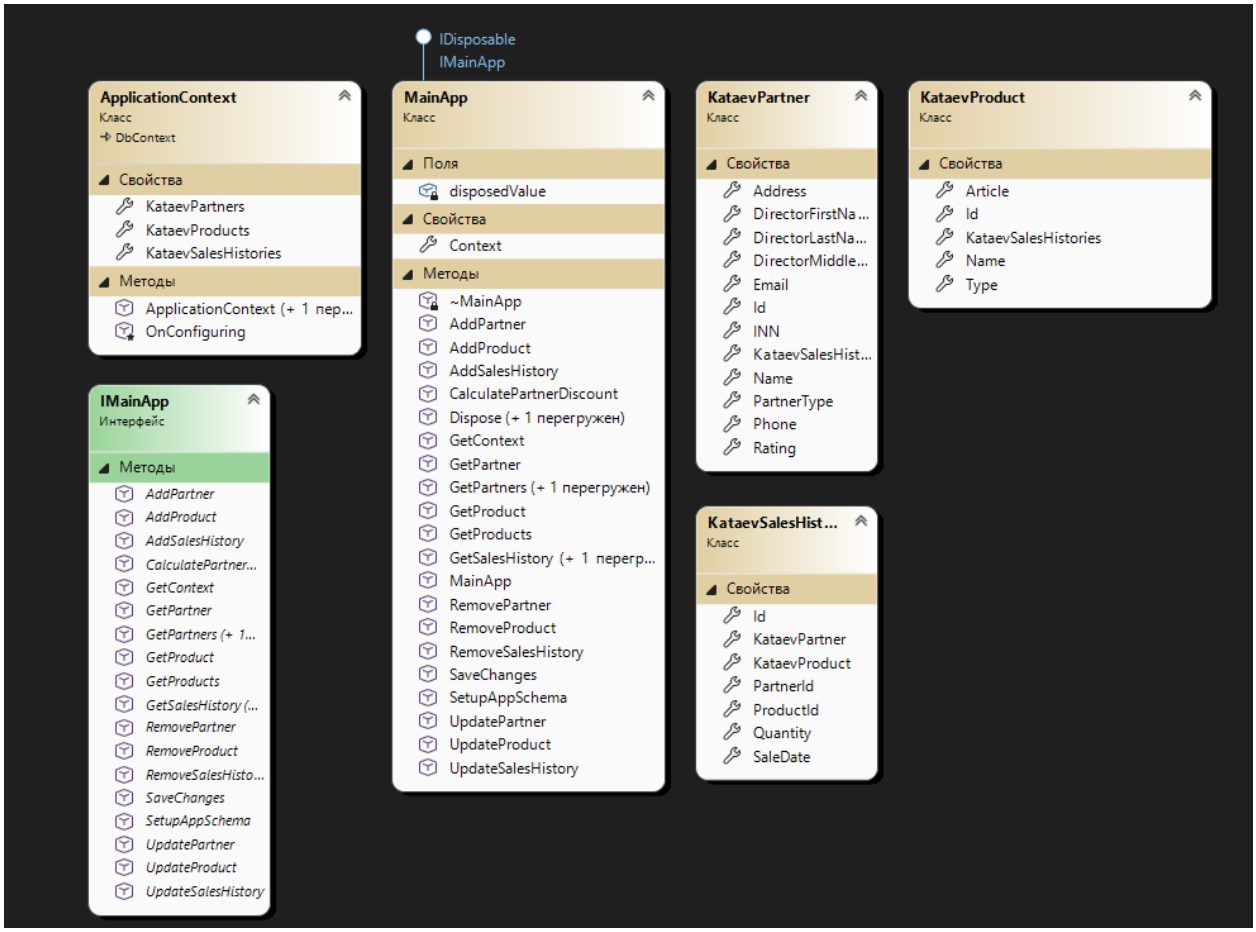

Рисунок 11 Диаграмма классов

Создание WPF приложения

После завершения разработки библиотеки классов KataevLIB создаётся второй проект в решении — WPF-приложение, которое будет служить пользовательским интерфейсом.

Создание проекта и подключение библиотеки

В решение добавляется новый проект типа «Приложение WPF (.NET Framework)». После его создания необходимо подключить библиотеку KataevLIB: нажимаем правой кнопкой мыши на раздел «Зависимости» WPF-проекта, выбираем «Добавить ссылку на проект» и отмечаем KataevLIB. После этого все классы и интерфейс библиотеки становятся доступны в WPF-приложении.

Главное окно MainWindow

Главное окно является основной точкой взаимодействия пользователя с системой. В его конструкторе создаётся экземпляр класса MainApp, через который осуществляются все операции с данными. Окно содержит список партнёров с отображением наименования и текущей скидки, панель с детальной информацией о выбранном партнёре и таблицу истории продаж. При выборе партнёра из списка автоматически загружаются его реквизиты и история продаж, а скидка пересчитывается и отображается на экране.

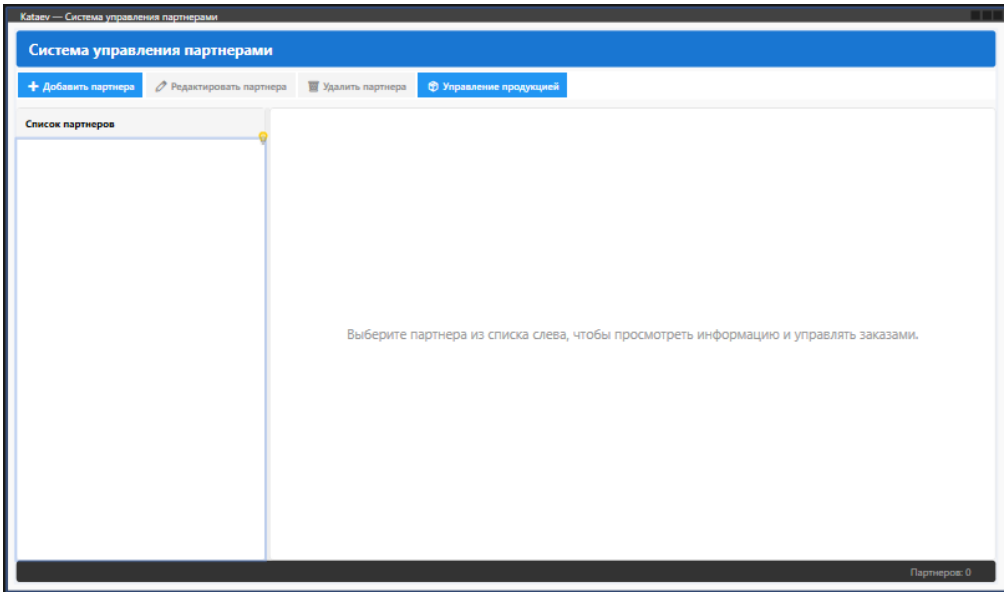

Рисунок 12 Главное окно MainWindow

Окно добавления и редактирования партнёра

Создаётся окно KataevPartnerEditWindow, содержащее поля ввода для всех реквизитов партнёра: наименования, типа организации, рейтинга, адреса, ИНН, фамилии, имени и отчества директора, телефона и электронной почты. При нажатии кнопки «Сохранить» данные передаются в MainApp, сохраняются в базе данных и список партнёров в главном окне обновляется.

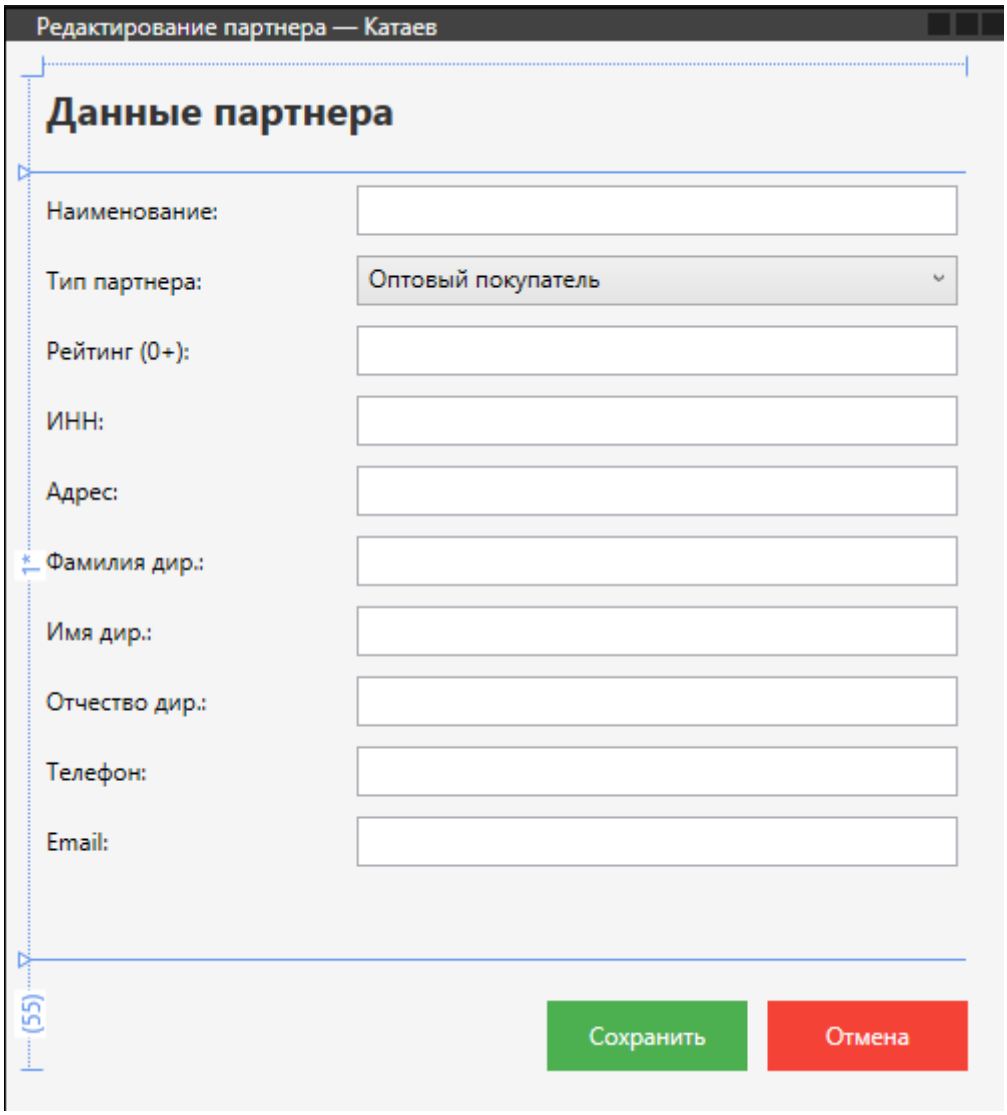

Рисунок 13 Окно добавления и редактирования партнёра

Окно управления продукцией

Создаётся окно KataevProductsWindow, реализующее полный цикл операций со справочником продукции: просмотр списка, добавление, редактирование и удаление позиций. Для добавления и редактирования открывается дочернее окно KataevProductEditWindow с полями для ввода наименования, артикула и типа продукции.

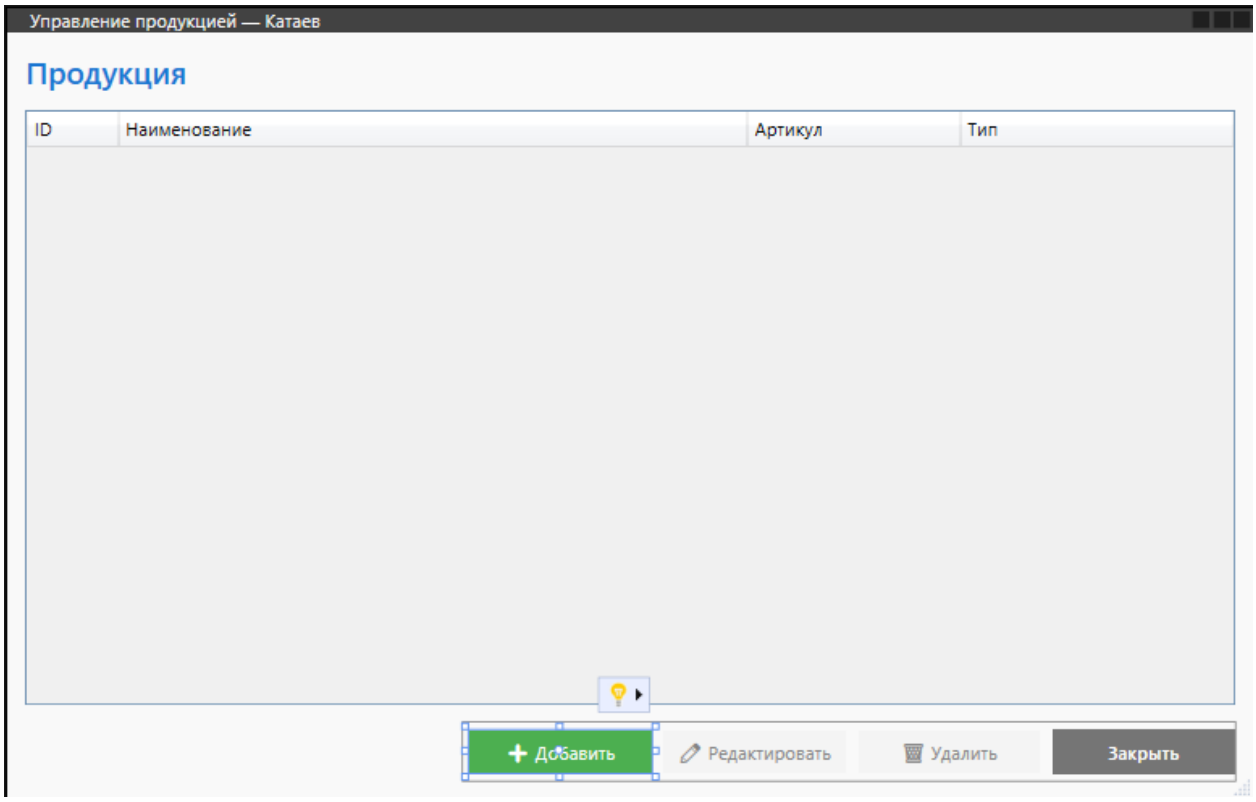

Рисунок 14 Окно управления продукцией

Окно добавления продажи

Создаётся окно KataevOrderEditWindow, позволяющее выбрать продукцию из справочника, указать количество реализованных единиц и дату продажи. После сохранения записи скидка партнёра в главном окне пересчитывается автоматически, отражая актуальный суммарный объём продаж.

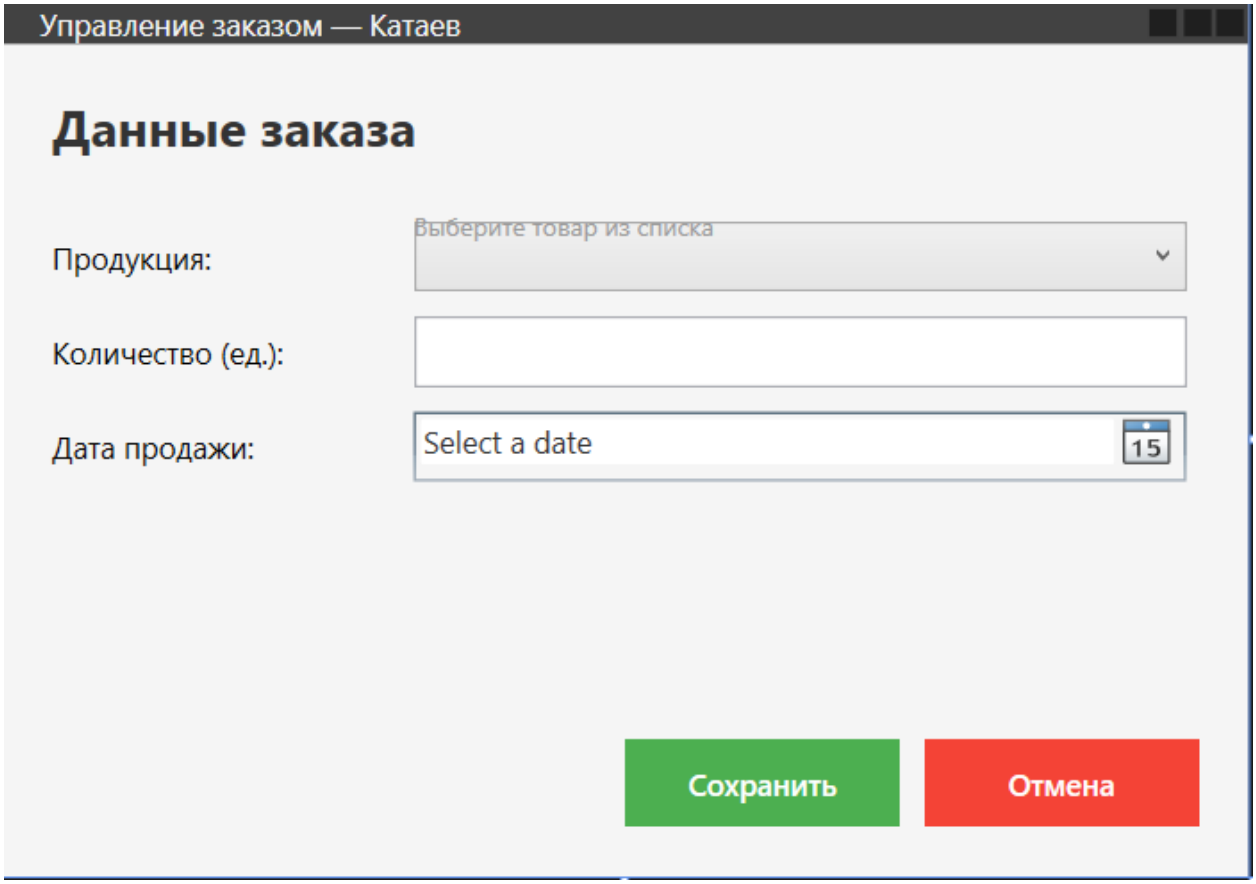

Рисунок 15 Окно добавления продажи

Стилизация интерфейса

Для обеспечения единого визуального стиля глобальные стили кнопок и элементов управления вынесены в файл App.xaml. Применены единые цвета, отступы и шрифты во всех окнах приложения, что соответствует принципам Material Design.

```
App.xaml
Application
1   <Application x:Class="KataevWPF.App"
2                xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3                xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4                StartupUri="MainWindow.xaml">
5       <Application.Resources>
6           <Style x:Key="ToolbarButton" TargetType="Button">
7               <Setter Property="Padding"      Value="10,6"/>
8               <Setter Property="Margin"       Value="2"/>
9               <Setter Property="Background"   Value="#2196F3"/>
10              <Setter Property="Foreground"   Value="White"/>
11              <Setter Property="BorderThickness" Value="0"/>
12              <Setter Property="Cursor"       Value="Hand"/>
13              <Setter Property="FontWeight"   Value="SemiBold"/>
14              <Setter Property="Height"       Value="30"/>
15              <Style.Triggers>
16                  <Trigger Property="IsMouseOver" Value="True">
17                      <Setter Property="Background" Value="#1976D2"/>
18                  </Trigger>
19                  <Trigger Property="IsEnabled" Value="False">
20                      <Setter Property="Background" Value="#BDBDBD"/>
21                  </Trigger>
22              </Style.Triggers>
23          </Style>
24
25          <Style x:Key="ActionButton" TargetType="Button">
26              <Setter Property="Padding"      Value="8,4"/>
27              <Setter Property="Margin"       Value="2"/>
28              <Setter Property="Background"   Value="#4CAF50"/>
29              <Setter Property="Foreground"   Value="White"/>
30              <Setter Property="BorderThickness" Value="0"/>
31              <Setter Property="Cursor"       Value="Hand"/>
32              <Setter Property="Height"       Value="28"/>
33              <Style.Triggers>
34                  <Trigger Property="IsMouseOver" Value="True">
35                      <Setter Property="Background" Value="#388E3C"/>
36                  </Trigger>
37                  <Trigger Property="IsEnabled" Value="False">
38                      <Setter Property="Background" Value="#BDBDBD"/>
39                  </Trigger>
40              </Style.Triggers>
41          </Style>
42
43          <BooleanToVisibilityConverter x:Key="BoolToVis"/>
44
45      </Application.Resources>
46  </Application>
```

Рисунок 16 файл App.xaml

Создание модульных тестов

Для проверки корректности бизнес-логики библиотеки KataevLIB был создан отдельный проект модульных тестов KataevLIB.Tests на платформе MSTest.

Создание тестового проекта

В решение добавляется новый проект типа «Тестовый проект MSTest (.NET Framework)». Аналогично WPF-приложению, в зависимости проекта добавляется ссылка на KataevLIB.

Подход к тестированию

Поскольку класс MainApp работает с реальной базой данных PostgreSQL, прямое его использование в тестах создаёт зависимость от внешней среды. Для решения этой проблемы был применён подход Mock-объектов: создан класс MockMainApp, реализующий интерфейс IMainApp через обычные коллекции List<T> в памяти. Это позволило тестировать логику полностью изолированно — без подключения к базе данных, быстро и воспроизводимо на любой машине.

Структура тестов

Тестовый класс MainAppTests содержит 34 теста, сгруппированных по функциональным областям.

Тесты операций с партнёрами проверяют добавление одного и нескольких партнёров, присвоение уникальных идентификаторов, поиск по наименованию, получение по идентификатору, обновление полей и удаление без влияния на остальные записи.

Тесты операций с продукцией охватывают добавление, получение списка и отдельной позиции, обновление и удаление.

Тесты истории продаж проверяют добавление записи при существующем партнёре, генерацию исключения при несуществующем партнёре, фильтрацию по идентификатору партнёра, обновление количества и удаление записи.

Тесты расчёта скидки проверяют все пороговые значения: менее 10 000 единиц возвращает 0%, ровно 10 000 и диапазон до 50 000 возвращает 5%, ровно 50 000 и диапазон до 300 000 возвращает 10%, ровно 300 000 и выше возвращает 15%. Отдельно проверяется суммирование количества по нескольким записям и нулевой результат при отсутствии продаж.

Запуск тестов

Тесты запускаются через встроенный в Visual Studio 2022 инструмент Test Explorer. Для его открытия выбирается меню «Тест» → «Обозреватель тестов». После сборки решения все тесты отображаются в списке и запускаются кнопкой «Выполнить все». Все 34 теста успешно пройдены, что подтверждает корректность реализованной бизнес-логики.

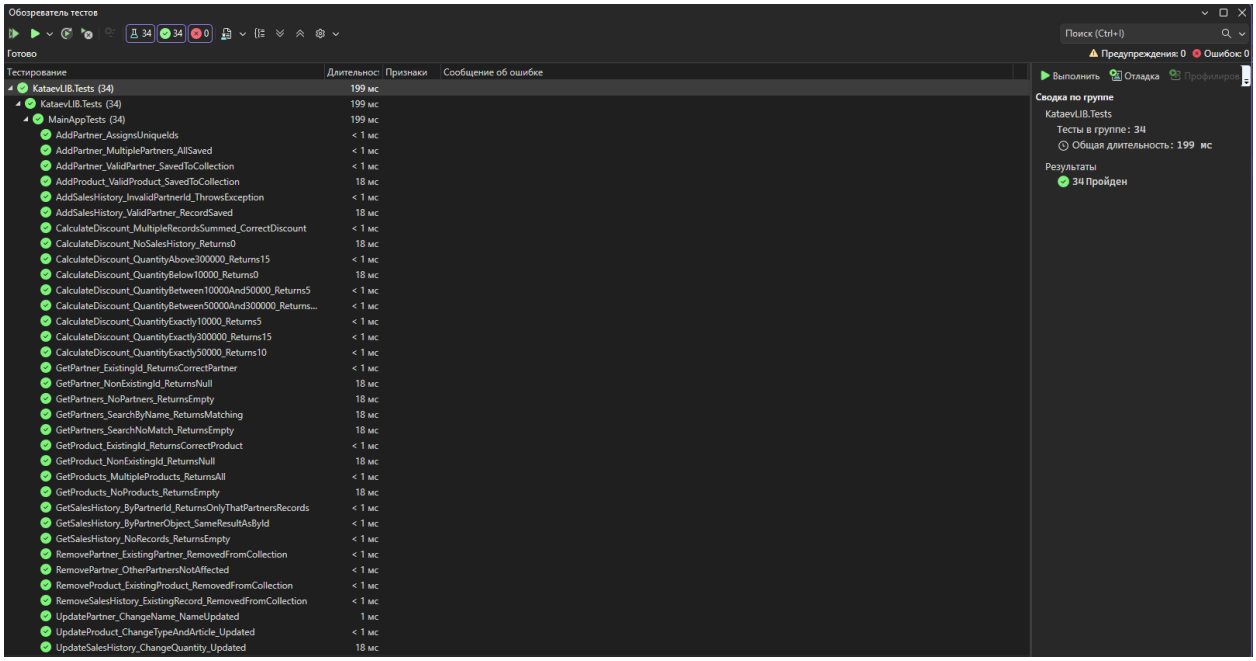

Рисунок 17 Запуск тестов

Вывод

В ходе выполнения данной работы был разработан программный продукт «Учёт партнёров и расчёт индивидуальных скидок» для производственной компании «Мастер пол».

В результате работы были достигнуты следующие результаты:

Спроектирована и реализована реляционная база данных в СУБД PostgreSQL 16, содержащая три взаимосвязанные таблицы в схеме app. Все объекты базы данных содержат префикс Kataev в соответствии с требованиями задания. Для обеспечения безопасности доступа создана отдельная роль app с минимально необходимыми правами.

Разработана библиотека классов KataevLIB, содержащая модели данных, контекст Entity Framework Core, интерфейс IMainApp и класс бизнес-логики MainApp. Подключение к базе данных реализовано через провайдер Npgsql. Ключевым методом бизнес-логики является CalculatePartnerDiscount, автоматически рассчитывающий индивидуальную скидку партнёра на основании суммарного объёма реализованной продукции.

Разработан пользовательский интерфейс на базе технологии WPF, обеспечивающий выполнение всех операций с данными: добавление, редактирование, удаление и просмотр партнёров, продукции и истории продаж.

Написан комплекс из 34 модульных тестов на платформе MSTest, покрывающих все методы интерфейса IMainApp. Все тесты успешно пройдены, что подтверждает корректность реализованной логики.

Исходный код проекта и сопроводительная записка размещены в репозитории GitHub.